Near ITPB, Channasandra, Bengaluru – 560067
Affiliated to VTU, Belagavi
Approved by AICTE, New Delhi
Recognized by UGC under 2(f) & 12(B)
Accredited by NBA & NAAC

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

(DATA SCIENCE)

VII SEMESTER

MVJ22CD61-DEEP LEARNING LAB

LABORATORY MANUAL

NAME OF THE STUDENT : _____

BRANCH : _____

UNIVERSITY SEAT NO. : _____

SEMESTER & SECTION : _____

BATCH : _____

VISION

The Department of Data Science aspires to become a school of excellence, providing quality education, constructive research, and professional opportunities in Data Science.

MISSION

- To provide academic programs that engage, enlighten and empower the students to learn technology through practice, service and outreach.
- To impart quality and value-based education, and contribute towards the innovation of computing, expert systems and Data Science, to the satisfaction of all stakeholders.
- To encourage research through continuous improvement in infrastructure, curriculum and faculty development, in collaboration with industry and institutions.

PROGRAM OUTCOMES (POs):

1. **PO1: Engineering Knowledge:** Apply knowledge of mathematics, natural science, computing, engineering fundamentals and an engineering specialization as specified in WK1 to WK4 respectively to develop to the solution of complex engineering problems.

2. **PO2: Problem Analysis:** Identify, formulate, review research literature and analyze complex engineering problems reaching substantiated conclusions with consideration for sustainable development. (WK1 to WK4)

3. **PO3: Design/Development of Solutions:** Design creative solutions for complex engineering problems and design/develop systems/components/processes to meet identified needs with consideration for the public health and safety, whole-life cost, net zero carbon, culture, society and environment as required. (WK5)

4. **PO4: Conduct Investigations of Complex Problems:** Conduct investigations of complex engineering problems using research-based knowledge including design of experiments, modelling, analysis & interpretation of data to provide valid conclusions. (WK8).

5. **PO5: Engineering Tool Usage:** Create, select and apply appropriate techniques, resources and modern engineering & IT tools, including prediction and modelling recognizing their limitations to solve complex engineering problems. (WK2 and WK6)

6. **PO6: The Engineer and The World:** Analyze and evaluate societal and environmental aspects while solving complex engineering problems for its impact on sustainability with reference to economy, health, safety, legal framework, culture and environment. (WK1, WK5, and WK7).

7. **PO7: Ethics:** Apply ethical principles and commit to professional ethics, human values, diversity and inclusion; adhere to national & international laws. (WK9)

8. **PO8: Individual and Collaborative Team work:** Function effectively as an individual, and as a member or leader in diverse/multi-disciplinary teams.

9. **PO9: Communication:** Communicate effectively and inclusively within the engineering community and society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations considering cultural, language, and learning differences.

10. **PO10: Project Management and Finance:** Apply knowledge and understanding of engineering management principles and economic decision-making and apply these to one's own work, as a member and leader in a team, and to manage projects and in multidisciplinary environments.

11. **PO11: Life-Long Learning:** Recognize the need for and have the preparation and ability for i) independent and life-long learning ii) adaptability to new and emerging technologies and iii) critical thinking in the broadest context of technological change.

PROGRAM EDUCATIONAL OBJECTIVES

PEO 1: Our Graduates will have the capability to apply their knowledge and the skills that they have acquired, to solve issues in real world Data Science sectors, and to develop feasible and viable systems.

PEO 2: Our Graduates will have the potential to participate in life-long learning through the successful completion of advanced degrees, continuing education, certifications and/or other professional developments.

PEO 3: Our Graduates will actively pursue post graduate studies in advanced areas of Data Science and related fields, by succeeding in competitive exams.

PROGRAM SPECIFIC OUTCOMES (PSOs):

PSO 1	Our Graduates will be endowed with the ability to become leaders in Industry and Academia, by possessing advanced knowledge and skills, which can empower them to analyze, design, develop and implement their learning to develop the society.
PSO 2	Our Graduates will have the ability to develop the skills to address and solve social and environmental problems in an ethical way, and perform multidisciplinary projects using advanced technologies and tools

COURSE OBJECTIVES:

COURSE OBJECTIVES: This course will enable students to

1.Understand the concepts and applications of neural networks and deep learning.
2.Understand CNN algorithms and the way to evaluate performance of the CNN architectures
3.Apply RNN and LSTM to learn, predict and classify the real-world problems in the paradigms of Deep Learning.
4.Understand, learn and design GANs for the selected problems.
5.Understand the concept of Auto-encoders and enhancing GANs using auto-encoders.

PREREQUISITES:

- Machine Learning Fundamentals.

SL NO	LIST OF PROGRAMS
1	Implement a Deep Feed Forward ANN (Multi-Layer Perceptron) using back propagation using a sequential dataset.
2	Implement a Deep Feed Forward ANN (Multi-Layer Perceptron) to classify images using MNIST datasets.
3	Implement a CNN model with 2 layers of convolution. Use padding and batch normalization.
4	Implement a CNN model with four layers of convolution and study the effect of Regularization and dropout. Specifically test the accuracy of the model for following scenarios. 1) Base model of CNN. 2) CNN Model with L1 Regularization. 3) CNN Model with L2 Regularization. 4) CNN Model with Dropout. 5) CNN Model with L2 (or L1) regularization and Dropout(combined).
5	Implement a python program to demonstrate the Image captioning.
6	Implement a text classification model using Bidirectional-LSTM on a suitable dataset.
7	Implement Auto encoders for image denoising on MNIST dataset.
8	Implement a Generative Adversarial Network using MNIST dataset to train a model that generates realistic handwritten digits.
9	Implement a Python program to build a model for face recognition using Python and Open-CV.
10	Implement an RNN for sentiment analysis using IMDB movie dataset.

COURSE OUTCOMES (CO's):

CO No	CO's
CO1	Understand the concepts and applications of neural networks and deep learning.
CO2	Understand CNN algorithms and the way to evaluate performance of the CNN architectures
CO3	Apply RNN and LSTM to learn, predict and classify the real-world problems in the paradigms of Deep Learning.
CO4	Understand, learn and design GANs for the selected problems.

CO5	Understand the concept of Auto-encoders and enhancing GANs using auto-encoders.

Mapping of Course Outcomes to Program Outcomes:

CO/PO	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12
CO1	3	2	3									
CO2	2	2	3									
CO3	1	1	2									
CO4	2	2	2									
CO5	3	1	2									

Level 3- Highly Mapped, Level 2-Moderately Mapped, Level 1-Low Mapped

Experiment 1.

Implement a Deep Feed Forward ANN (Multi-Layer Perceptron) using back propagation using a sequential dataset.

```
from keras.models import Sequential
from keras.layers import Dense, Activation
import numpy as np
import pandas as pd
from sklearn import datasets
iris = datasets.load_iris()
X, y = datasets.load_iris(return_X_y = True)
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.40)
# Define the network model and its arguments.
# Set the number of neurons/nodes for each layer:
model = Sequential()
model.add(Dense(2, input_shape=(4,)))
model.add(Activation('sigmoid'))
model.add(Dense(1))
model.add(Activation('sigmoid'))
model.add(Dense(2, input_shape=(4,)))
model.add(Activation('sigmoid'))
model.add(Dense(1))
model.add(Activation('sigmoid'))
model.add(Dense(2, input_shape=(4,)))
model.add(Activation('sigmoid'))
model.add(Dense(1))
model.add(Activation('sigmoid'))
#sgd = SGD(lr=0.0001, decay=1e-6, momentum=0.9, nesterov=True)
#model.compile(loss='categorical_crossentropy', optimizer=sgd, metrics=['accuracy'])
# Compile the model and calculate its accuracy:
model.compile(loss='mean_squared_error', optimizer='sgd', metrics=['accuracy'])
#model.fit(X_train, y_train, batch_size=32, epochs=3)
# Print a summary of the Keras model:
model.summary()
#model.fit(X_train, y_train)
#model.fit(X_train, y_train, batch_size=32, epochs=300)
model.fit(X_train, y_train, epochs=5)
score = model.evaluate(X_test, y_test)
print(score)
```

Output :

Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 2)	10
activation (Activation)	(None, 2)	0
dense_1 (Dense)	(None, 1)	3
activation_1 (Activation)	(None, 1)	0
dense_2 (Dense)	(None, 2)	4
activation_2 (Activation)	(None, 2)	0
dense_3 (Dense)	(None, 1)	3
activation_3 (Activation)	(None, 1)	0
dense_4 (Dense)	(None, 2)	4
activation_4 (Activation)	(None, 2)	0
dense_5 (Dense)	(None, 1)	3
activation_5 (Activation)	(None, 1)	0

Total params: 27 (108.00 B)
Trainable params: 27 (108.00 B)
Non-trainable params: 0 (0.00 B)

Total params: 27 (108.00 B)
Trainable params: 27 (108.00 B)
Non-trainable params: 0 (0.00 B)
Epoch 1/5
3/3 ━━━━━━━━━━━━━━━━━━━━ 2s 41ms/step - accuracy: 0.3379 - loss: 0.9622
Epoch 2/5
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 32ms/step - accuracy: 0.3653 - loss: 0.8832
Epoch 3/5
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 41ms/step - accuracy: 0.3692 - loss: 0.9534
Epoch 4/5
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 25ms/step - accuracy: 0.3536 - loss: 0.9761
Epoch 5/5
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - accuracy: 0.3614 - loss: 0.8997
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 57ms/step - accuracy: 0.2833 - loss: 1.0733
[1.023298978805542, 0.30000001192092896]

Experiment 2.

Implement a Deep Feed Forward ANN (Multi-Layer Perceptron) to classify images using MNIST datasets.

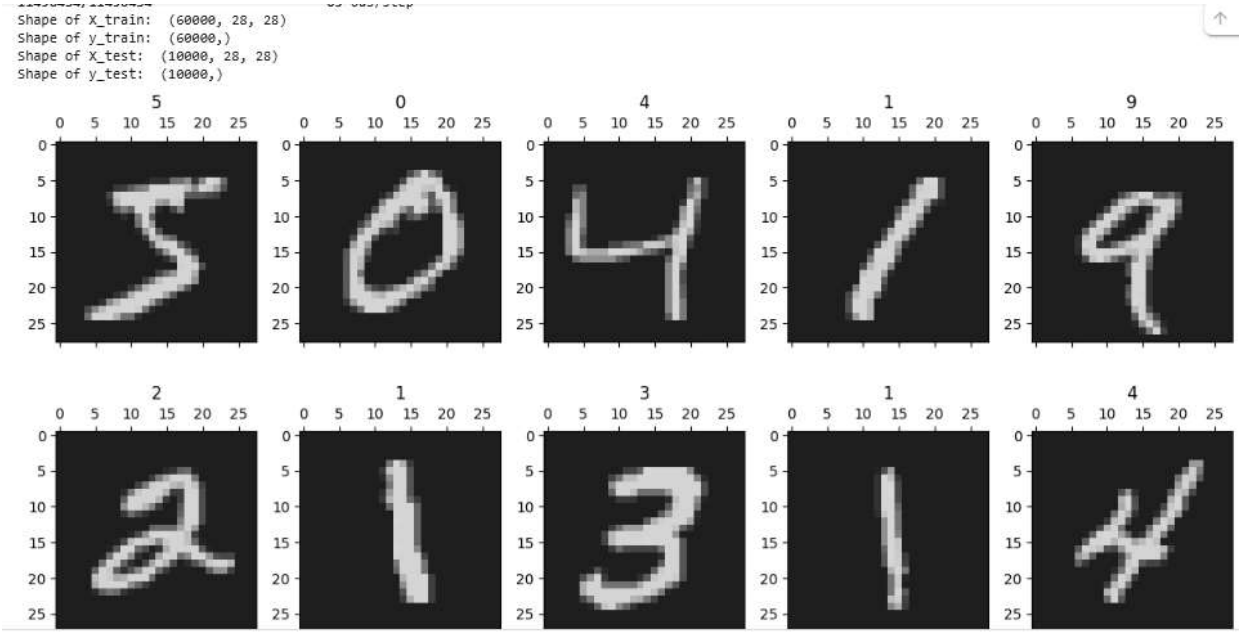
```
import tensorflow as tf
from tensorflow import keras
from keras.models import Sequential
from keras import Input
from keras.layers import Dense
import pandas as pd
import numpy as np
import sklearn
from sklearn.metrics import classification_report
import matplotlib
import matplotlib.pyplot as plt
# Load digits data
(X_train, y_train), (X_test, y_test) = keras.datasets.mnist.load_data()
# Print shapes
print("Shape of X_train: ", X_train.shape)
print("Shape of y_train: ", y_train.shape)
print("Shape of X_test: ", X_test.shape)
print("Shape of y_test: ", y_test.shape)
# Display images of the first 10 digits in the training set and their true lables
fig, axs = plt.subplots(2, 5, sharey=False, tight_layout=True, figsize=(12,6),
facecolor='white')
n=0
for i in range(0,2):
    for j in range(0,5):
        axs[i,j].matshow(X_train[n])
        axs[i,j].set(title=y_train[n])
        n=n+1
plt.show()
# Reshape and normalize (divide by 255) input data
X_train = X_train.reshape(60000, 784).astype("float32") / 255
X_test = X_test.reshape(10000, 784).astype("float32") / 255
# Print shapes
print("New shape of X_train: ", X_train.shape)
print("New shape of X_test: ", X_test.shape)
#Design the Deep FF Neural Network architecture
model = Sequential(name="DFF-Model") # Model
model.add(Input(shape=(784,), name='Input-Layer')) # Input Layer - need to specify the
shape of inputsR-2
model.add(Dense(128, activation='relu', name='Hidden-Layer-1',
kernel_initializer='HeNormal'))
```

```
model.add(Dense(64, activation='relu', name='Hidden-Layer-2',
kernel_initializer='HeNormal'))
model.add(Dense(32, activation='relu', name='Hidden-Layer-3',
kernel_initializer='HeNormal'))
model.add(Dense(10, activation='softmax', name='Output-Layer'))
#Compile keras model
model.compile(optimizer='adam',
            loss='SparseCategoricalCrossentropy',
            metrics=['Accuracy'])
#             loss_weights=None,
#             weighted_metrics=None,
#             run_eagerly=None,
#             steps_per_execution=None)
#Fit keras model on the dataset
model.fit(
      X_train,
      y_train,
      batch_size=10,
      epochs=5,
      verbose='auto',
      callbacks=None)
#       validation_split=0.2,
#       shuffle=True,
#       class_weight=None,
#       sample_weight=None,
#       initial_epoch=0,
#       steps_per_epoch=None,
#       validation_steps=None,
#       validation_batch_size=None,
#       validation_freq=1,)
#       max_q_size=10,
#       workers=1,
#       use_multiprocessing=False,)

#apply the trained model to make predictions
# Predict class labels on training data
pred_labels_tr = np.array(tf.math.argmax(model.predict(X_train),axis=1))
# Predict class labels on a test data
pred_labels_te = np.array(tf.math.argmax(model.predict(X_test),axis=1))
#Model Performance Summary
print("")
print(' Model Summary ')
model.summary()
print("")
```

```
# Printing the parameters:Deep Feed Forward Neural Network contains more than 100K
#print(' Weights and Biases
#for layer in model_d1.layers:
#print("Layer: ", layer.name) # print layer name
#print(" --Kernels (Weights): ", layer.get_weights()[0]) # kernels (weights)
#print(" --Biases: ", layer.get_weights()[1]) # biases
print("")
print('---------- Evaluation on Training Data ----------')
print(classification_report(y_train, pred_labels_tr))
print("")
print('---------- Evaluation on Test Data ----------')
print(classification_report(y_test, pred_labels_te))
print("")
```

Output :




```
New shape of X_train:  (60000, 784)
New shape of X_test:  (10000, 784)
Epoch 1/5
6000/6000 ━━━━━━━━━━━━━━━━━━━━ 25s 3ms/step - Accuracy: 0.8870 - loss: 0.3688
Epoch 2/5
6000/6000 ━━━━━━━━━━━━━━━━━━━━ 19s 3ms/step - Accuracy: 0.9656 - loss: 0.1118
Epoch 3/5
6000/6000 ━━━━━━━━━━━━━━━━━━━━ 19s 3ms/step - Accuracy: 0.9765 - loss: 0.0770
Epoch 4/5
6000/6000 ━━━━━━━━━━━━━━━━━━━━ 19s 3ms/step - Accuracy: 0.9809 - loss: 0.0619
Epoch 5/5
6000/6000 ━━━━━━━━━━━━━━━━━━━━ 18s 3ms/step - Accuracy: 0.9838 - loss: 0.0515
```

1875/1875 ━━━━━━━━━━━━━━━━━━━━ 3s 1ms/step

313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step

Model Summary

Model: "DFF-Model"

Layer (type)	Output Shape	Param #
Hidden-Layer-1 (Dense)	(None, 128)	100,480
Hidden-Layer-2 (Dense)	(None, 64)	8,256
Hidden-Layer-3 (Dense)	(None, 32)	2,080
Output-Layer (Dense)	(None, 10)	330

Total params: 333,440 (1.27 MB)
Trainable params: 111,146 (434.16 KB)
Non-trainable params: 0 (0.00 B)
Optimizer params: 222,294 (868.34 KB)

---------- Evaluation on Training Data ----------

	precision	recall	f1-score	support
0	0.99	0.99	0.99	5923
1	0.99	0.99	0.99	6742
2	0.98	0.99	0.98	5958
3	0.99	0.98	0.98	6131
4	0.98	1.00	0.99	5842
5	0.98	0.99	0.98	5421
6	0.98	1.00	0.99	5918
7	0.99	0.99	0.99	6265
8	0.99	0.97	0.98	5851
9	0.99	0.97	0.98	5949
accuracy			0.99	60000
macro avg	0.99	0.99	0.99	60000
weighted avg	0.99	0.99	0.99	60000

---------- Evaluation on Test Data ----------

	precision	recall	f1-score	support
0	0.98	0.99	0.98	980
1	0.99	0.99	0.99	1135
2	0.97	0.97	0.97	1032
3	0.98	0.97	0.97	1010
4	0.97	0.98	0.97	982
5	0.96	0.98	0.97	892
6	0.96	0.98	0.97	958
7	0.97	0.97	0.97	1028
8	0.98	0.94	0.96	974
9	0.97	0.96	0.97	1009
accuracy			0.97	10000
macro avg	0.97	0.97	0.97	10000
weighted avg	0.97	0.97	0.97	10000

Experiment 3.

Implement a CNN model with 2 layers of convolution. Use padding and batch normalization.

```
# Batch-Normalization and padding
import keras
from keras.datasets import fashion_mnist
from keras.layers import Dense, Activation, Flatten, Conv2D, MaxPooling2D,BatchNormalization
from keras.models import Sequential
from keras.utils import to_categorical
import numpy as np
import matplotlib.pyplot as plt
(train_X,train_Y), (test_X,test_Y) = fashion_mnist.load_data()
train_X = train_X.reshape(-1, 28,28, 1)
test_X = test_X.reshape(-1, 28,28, 1)
train_X = train_X.astype('float32')
test_X = test_X.astype('float32')
train_X = train_X / 255
test_X = test_X / 255
train_Y_one_hot = to_categorical(train_Y)
test_Y_one_hot = to_categorical(test_Y)
model = Sequential()
model.add(Conv2D(256, (3,3), input_shape=(28, 28, 1),padding='same'))
model.add(Activation('relu'))
BatchNormalization()
model.add(MaxPooling2D(pool_size=(2,2) ,padding='same'))
model.add(Conv2D(128, (3,3),padding='same'))
model.add(Activation('relu'))
#BatchNormalization()
model.add(MaxPooling2D(pool_size=(2,2) ,padding='same'))
model.add(Conv2D(64, (3,3), input_shape=(28, 28, 1),padding='same'))
model.add(Activation('relu'))
#BatchNormalization()
model.add(MaxPooling2D(pool_size=(2,2),padding='same'))
model.add(Conv2D(28, (3,3)))
model.add(Activation('relu'))
```

```
#BatchNormalization()
model.add(MaxPooling2D(pool_size=(2,2),padding='same'))
model.add(Flatten())
model.add(Dense(64))
model.add(Dense(10))
model.add(Activation('softmax'))
model.compile(loss=keras.losses.categorical_crossentropy,
optimizer=keras.optimizers.Adam(),metrics=['accuracy'])
model.fit(train_X, train_Y_one_hot, batch_size=64, epochs=5)
test_loss, test_acc = model.evaluate(test_X, test_Y_one_hot)
print('Test loss', test_loss)
print('Test accuracy', test_acc)
predictions = model.predict(test_X)
print(np.argmax(np.round(predictions[0])))
plt.imshow(test_X[0].reshape(28, 28), cmap = plt.cm.binary)
plt.show()
```

Output:

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/train-labels-idx1-ubyte.gz
29515/29515 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/train-images-idx3-ubyte.gz
26421880/26421880 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/t10k-labels-idx1-ubyte.gz
5148/5148 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/t10k-images-idx3-ubyte.gz
4422102/4422102 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
/usr/local/lib/python3.11/dist-packages/keras/src/layers/convolutional/base_conv.py:107: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Epoch 1/5
938/938 ━━━━━━━━━━━━━━━━━━━━ 690s 731ms/step - accuracy: 0.7273 - loss: 0.7388
Epoch 2/5
938/938 ━━━━━━━━━━━━━━━━━━━━ 751s 741ms/step - accuracy: 0.8823 - loss: 0.3280
Epoch 3/5
938/938 ━━━━━━━━━━━━━━━━━━━━ 733s 731ms/step - accuracy: 0.9014 - loss: 0.2733
Epoch 4/5
938/938 ━━━━━━━━━━━━━━━━━━━━ 739s 728ms/step - accuracy: 0.9170 - loss: 0.2296
Epoch 5/5
938/938 ━━━━━━━━━━━━━━━━━━━━ 682s 727ms/step - accuracy: 0.9256 - loss: 0.2049
313/313 ━━━━━━━━━━━━━━━━━━━━ 29s 92ms/step - accuracy: 0.9064 - loss: 0.2548
Test loss 0.2541674077510834
Test accuracy 0.9071000218391418
313/313 ━━━━━━━━━━━━━━━━━━━━ 30s 95ms/step
```

Experiment 4.

Implement a CNN model with four layers of convolution and study the effect of Regularization and dropout. Specifically test the accuracy of the model for following scenarios. 1) Base model of CNN. 2) CNN Model with L1 Regularization. 3) CNN Model with L2 Regularization. 4) CNN Model with Dropout. 5) CNN Model with L2 (or L1) regularization and Dropout (combined).

1) Base Version

```
import numpy as np
import matplotlib.pyplot as plt
from keras.datasets import fashion_mnist
from keras.models import Sequential
from keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Activation
from keras.utils import to_categorical

# Load and preprocess the data
(train_X, train_Y), (test_X, test_Y) = fashion_mnist.load_data()

# Reshape and normalize
train_X = train_X.reshape(-1, 28, 28, 1).astype('float32') / 255.0
test_X = test_X.reshape(-1, 28, 28, 1).astype('float32') / 255.0

# One-hot encode the labels
train_Y_one_hot = to_categorical(train_Y, 10)
test_Y_one_hot = to_categorical(test_Y, 10)

# Build CNN model without regularization and dropout
model = Sequential()
```

```
# 1st Conv Layer
model.add(Conv2D(32, (3, 3), padding='same', input_shape=(28, 28, 1)))
model.add(Activation('relu'))
model.add(MaxPooling2D(pool_size=(2, 2)))

# 2nd Conv Layer
model.add(Conv2D(64, (3, 3), padding='same'))
model.add(Activation('relu'))
model.add(MaxPooling2D(pool_size=(2, 2)))

# 3rd Conv Layer
model.add(Conv2D(128, (3, 3), padding='same'))
model.add(Activation('relu'))

# 4th Conv Layer
model.add(Conv2D(128, (3, 3), padding='same'))
model.add(Activation('relu'))

# Flatten and Dense layers
model.add(Flatten())
model.add(Dense(128, activation='relu'))
model.add(Dense(10, activation='softmax'))

# Compile the model
model.compile(optimizer='adam',
              loss='categorical_crossentropy',
              metrics=['accuracy'])

# Train the model
model.fit(train_X, train_Y_one_hot, epochs=10, batch_size=64, validation_split=0.1)

# Evaluate the model
test_loss, test_acc = model.evaluate(test_X, test_Y_one_hot)
print("Test Loss:", test_loss)
print("Test Accuracy:", test_acc)

# Predict and visualize
predictions = model.predict(test_X)
plt.imshow(test_X[0].reshape(28, 28), cmap='gray')
plt.title(f"Predicted: {np.argmax(predictions[0])}")
plt.axis('off')
plt.show()
```

Output :

Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/train-labels-idx1-ubyte.gz

29515/29515 ━━━━━━━━━━━━━━━━━━━━ **0s** 0us/step

Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/train-images-idx3-ubyte.gz

26421880/26421880 ━━━━━━━━━━━━━━━━━━━━ **0s** 0us/step

Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/t10k-labels-idx1-ubyte.gz

5148/5148 ━━━━━━━━━━━━━━━━━━━━ **0s** 0us/step

Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/t10k-images-idx3-ubyte.gz

4422102/4422102 ━━━━━━━━━━━━━━━━━━━━ **0s** 0us/step

/usr/local/lib/python3.11/dist-packages/keras/src/layers/convolutional/base_conv.py:107: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)

Epoch 1/10

844/844 ━━━━━━━━━━━━━━━━━━━━ **175s** 200ms/step - accuracy: 0.7631 - loss: 0.6413 - val_accuracy: 0.8907 - val_loss: 0.2969

Epoch 2/10

844/844 ━━━━━━━━━━━━━━━━━━━━ **170s** 201ms/step - accuracy: 0.8965 - loss: 0.2807 - val_accuracy: 0.9072 - val_loss: 0.2479

Epoch 3/10

844/844 ━━━━━━━━━━━━━━━━━━━━ **195s** 194ms/step - accuracy: 0.9162 - loss: 0.2278 - val_accuracy: 0.9117 - val_loss: 0.2339

Epoch 4/10

844/844 ━━━━━━━━━━━━━━━━━━━━ **205s** 197ms/step - accuracy: 0.9252 - loss: 0.2001 - val_accuracy: 0.9098 - val_loss: 0.2468

Epoch 5/10

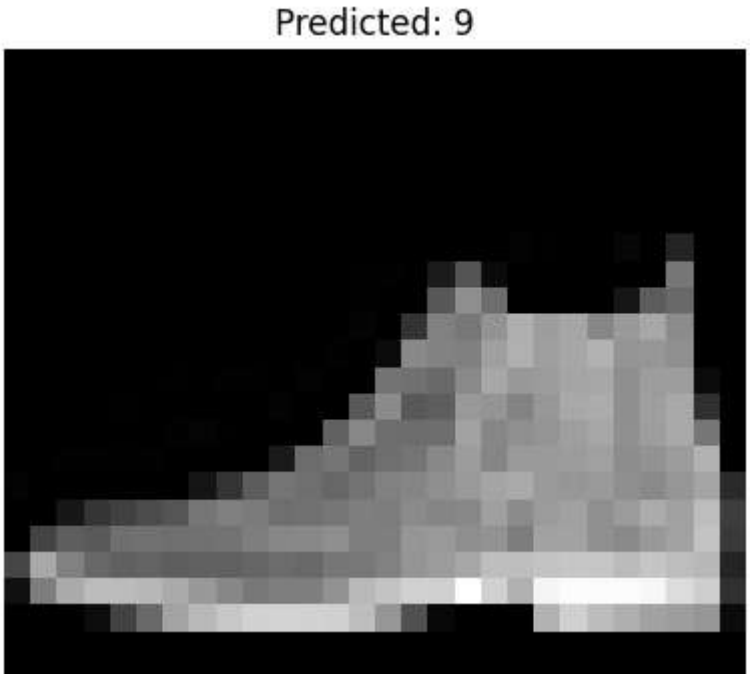
844/844 ━━━━━━━━━━━━━━━━━━━━ **199s** 194ms/step - accuracy: 0.9385 - loss: 0.1651 - val_accuracy: 0.9192 - val_loss: 0.2272

Epoch 6/10

844/844 ━━━━━━━━━━━━━━━━━━━━ **162s** 192ms/step - accuracy: 0.9475 - loss: 0.1455 - val_accuracy: 0.9173 - val_loss: 0.2168

Epoch 7/10

844/844 ━━━━━━━━━━━━━━━━━━━━ **167s** 197ms/step - accuracy: 0.9579 - loss: 0.1157 - val_accuracy: 0.9195 - val_loss: 0.2471
Epoch 8/10
844/844 ━━━━━━━━━━━━━━━━━━━━ **202s** 198ms/step - accuracy: 0.9627 - loss: 0.0986 - val_accuracy: 0.9197 - val_loss: 0.2380
Epoch 9/10
844/844 ━━━━━━━━━━━━━━━━━━━━ **172s** 203ms/step - accuracy: 0.9706 - loss: 0.0805 - val_accuracy: 0.9177 - val_loss: 0.2809
Epoch 10/10
844/844 ━━━━━━━━━━━━━━━━━━━━ **203s** 205ms/step - accuracy: 0.9769 - loss: 0.0642 - val_accuracy: 0.9185 - val_loss: 0.2697
313/313 ━━━━━━━━━━━━━━━━━━━━ **9s** 29ms/step - accuracy: 0.9198 - loss: 0.3011
Test Loss: 0.2918735146522522
Test Accuracy: 0.9217000007629395
313/313 ━━━━━━━━━━━━━━━━━━━━ **9s** 29ms/step



Experiment 4.

2) CNN Model with L1 Regularization

```
import numpy as np
import matplotlib.pyplot as plt
from keras.datasets import fashion_mnist
from keras.models import Sequential
from keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Activation, Dropout
from keras.utils import to_categorical
from keras import regularizers

# Load and preprocess the data
(train_X, train_Y), (test_X, test_Y) = fashion_mnist.load_data()

# Reshape to fit the model and normalize
train_X = train_X.reshape(-1, 28, 28, 1).astype('float32') / 255.0
test_X = test_X.reshape(-1, 28, 28, 1).astype('float32') / 255.0

# One-hot encode the labels
train_Y_one_hot = to_categorical(train_Y, 10)
test_Y_one_hot = to_categorical(test_Y, 10)

# L1 regularization strength
l1_strength = 0.001

# Build CNN model with 4 Conv layers
model = Sequential()

# 1st Conv Layer
model.add(Conv2D(32, (3,3), padding='same', input_shape=(28, 28, 1),
          kernel_regularizer=regularizers.l1(l1_strength)))
model.add(Activation('relu'))
model.add(MaxPooling2D(pool_size=(2,2)))

# 2nd Conv Layer
model.add(Conv2D(64, (3,3), padding='same',
          kernel_regularizer=regularizers.l1(l1_strength)))
model.add(Activation('relu'))
model.add(MaxPooling2D(pool_size=(2,2)))

# 3rd Conv Layer
model.add(Conv2D(128, (3,3), padding='same',
          kernel_regularizer=regularizers.l1(l1_strength)))
model.add(Activation('relu'))

# 4th Conv Layer
model.add(Conv2D(128, (3,3), padding='same',
```

```
            kernel_regularizer=regularizers.l1(l1_strength)))
model.add(Activation('relu'))

# Flatten and Dense layers
model.add(Flatten())
model.add(Dense(128, activation='relu',
            kernel_regularizer=regularizers.l1(l1_strength)))
model.add(Dropout(0.5))  # Optional dropout for better generalization
model.add(Dense(10, activation='softmax'))

# Compile the model
model.compile(optimizer='adam',
            loss='categorical_crossentropy',
            metrics=['accuracy'])

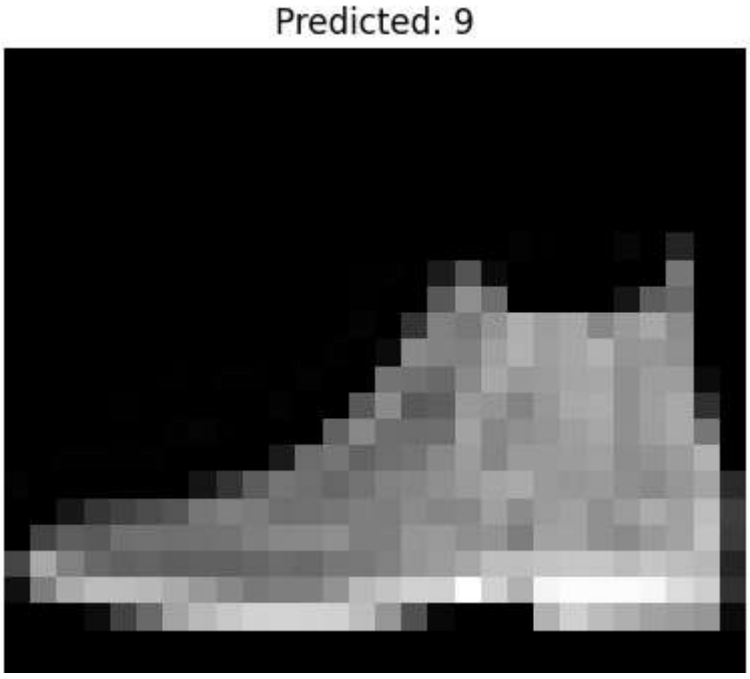
# Train the model
model.fit(train_X, train_Y_one_hot, epochs=10, batch_size=64, validation_split=0.1)

# Evaluate the model
test_loss, test_acc = model.evaluate(test_X, test_Y_one_hot)
print("Test Loss:", test_loss)
print("Test Accuracy:", test_acc)

# Predict and display a sample
predictions = model.predict(test_X)
plt.imshow(test_X[0].reshape(28,28), cmap='gray')
plt.title(f"Predicted: {np.argmax(predictions[0])}")
plt.axis('off')
plt.show()
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/train-labels-idx1-ubyte.gz
29515/29515 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/train-images-idx3-ubyte.gz
26421880/26421880 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/t10k-labels-idx1-ubyte.gz
5148/5148 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/t10k-images-idx3-ubyte.gz
4422102/4422102 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
/usr/local/lib/python3.11/dist-packages/keras/src/layers/convolutional/base_conv.py:107: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Epoch 1/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 167s 194ms/step - accuracy: 0.5749 - loss: 3.4150 - val_accuracy: 0.7917 - val_loss: 0.9251
Epoch 2/10
```

844/844 ━━━━━━━━━━━━━━━━━━━━ 201s 193ms/step - accuracy: 0.7674 - loss: 1.0121 - val_accuracy: 0.8203 - val_loss: 0.8386
Epoch 3/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 201s 192ms/step - accuracy: 0.7874 - loss: 0.9139 - val_accuracy: 0.8202 - val_loss: 0.7839
Epoch 4/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 202s 193ms/step - accuracy: 0.8010 - loss: 0.8695 - val_accuracy: 0.8292 - val_loss: 0.7757
Epoch 5/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 202s 193ms/step - accuracy: 0.8089 - loss: 0.8384 - val_accuracy: 0.8430 - val_loss: 0.7333
Epoch 6/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 163s 193ms/step - accuracy: 0.8126 - loss: 0.8264 - val_accuracy: 0.8427 - val_loss: 0.7509
Epoch 7/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 168s 199ms/step - accuracy: 0.8173 - loss: 0.8074 - val_accuracy: 0.8432 - val_loss: 0.7085
Epoch 8/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 196s 192ms/step - accuracy: 0.8208 - loss: 0.7969 - val_accuracy: 0.8483 - val_loss: 0.7100
Epoch 9/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 163s 193ms/step - accuracy: 0.8245 - loss: 0.7893 - val_accuracy: 0.8490 - val_loss: 0.7027
Epoch 10/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 203s 194ms/step - accuracy: 0.8286 - loss: 0.7802 - val_accuracy: 0.8510 - val_loss: 0.6911
313/313 ━━━━━━━━━━━━━━━━━━━━ 9s 30ms/step - accuracy: 0.8487 - loss: 0.7028
Test Loss: 0.7045672535896301
Test Accuracy: 0.8481000065803528
313/313 ━━━━━━━━━━━━━━━━━━━━ 9s 28ms/step



Experiment 4.

3) CNN Model with L2 Regularization

```
import numpy as np
import matplotlib.pyplot as plt
from keras.datasets import fashion_mnist
from keras.models import Sequential
from keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Activation
from keras.utils import to_categorical
from keras import regularizers

# Load and preprocess the data
(train_X, train_Y), (test_X, test_Y) = fashion_mnist.load_data()
train_X = train_X.reshape(-1, 28, 28, 1).astype('float32') / 255.0
test_X = test_X.reshape(-1, 28, 28, 1).astype('float32') / 255.0

# One-hot encode the labels
train_Y_one_hot = to_categorical(train_Y, 10)
test_Y_one_hot = to_categorical(test_Y, 10)

# Set L2 regularization strength
l2_strength = 0.001

# Build CNN model with L2 regularization
model = Sequential()

# 1st Conv Layer
model.add(Conv2D(32, (3, 3), padding='same', input_shape=(28, 28, 1),
          kernel_regularizer=regularizers.l2(l2_strength)))
model.add(Activation('relu'))
model.add(MaxPooling2D(pool_size=(2, 2)))

# 2nd Conv Layer
model.add(Conv2D(64, (3, 3), padding='same',
          kernel_regularizer=regularizers.l2(l2_strength)))
model.add(Activation('relu'))
model.add(MaxPooling2D(pool_size=(2, 2)))

# 3rd Conv Layer
model.add(Conv2D(128, (3, 3), padding='same',
```

```
            kernel_regularizer=regularizers.l2(l2_strength)))
model.add(Activation('relu'))

# 4th Conv Layer
model.add(Conv2D(128, (3, 3), padding='same',
            kernel_regularizer=regularizers.l2(l2_strength)))
model.add(Activation('relu'))

# Flatten and Dense layers
model.add(Flatten())
model.add(Dense(128, activation='relu',
            kernel_regularizer=regularizers.l2(l2_strength)))
model.add(Dense(10, activation='softmax'))

# Compile the model
model.compile(optimizer='adam',
            loss='categorical_crossentropy',
            metrics=['accuracy'])

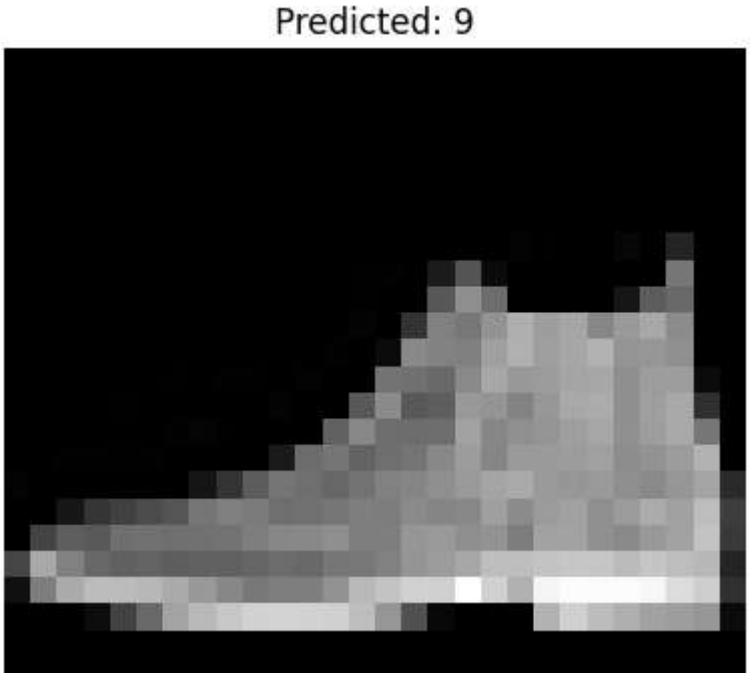
# Train the model
model.fit(train_X, train_Y_one_hot, epochs=10, batch_size=64, validation_split=0.1)

# Evaluate the model
test_loss, test_acc = model.evaluate(test_X, test_Y_one_hot)
print("Test Loss:", test_loss)
print("Test Accuracy:", test_acc)

# Predict and visualize
predictions = model.predict(test_X)
plt.imshow(test_X[0].reshape(28, 28), cmap='gray')
plt.title(f"Predicted: {np.argmax(predictions[0])}")
plt.axis('off')
plt.show()
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/train-labels-idx1-ubyte.gz
29515/29515 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/train-images-idx3-ubyte.gz
26421880/26421880 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/t10k-labels-idx1-ubyte.gz
5148/5148 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/t10k-images-idx3-ubyte.gz
4422102/4422102 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
/usr/local/lib/python3.11/dist-packages/keras/src/layers/convolutional/base_conv.py:107: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
```

```
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Epoch 1/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 169s 195ms/step - accuracy: 0.7651 - loss: 0.9108 - val_accuracy: 0.8755 - val_loss:
0.4870
Epoch 2/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 199s 192ms/step - accuracy: 0.8785 - loss: 0.4738 - val_accuracy: 0.8793 - val_loss:
0.4428
Epoch 3/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 208s 198ms/step - accuracy: 0.8895 - loss: 0.4267 - val_accuracy: 0.8908 - val_loss:
0.4165
Epoch 4/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 197s 193ms/step - accuracy: 0.8976 - loss: 0.3932 - val_accuracy: 0.8915 - val_loss:
0.3997
Epoch 5/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 202s 193ms/step - accuracy: 0.9023 - loss: 0.3785 - val_accuracy: 0.8987 - val_loss:
0.3952
Epoch 6/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 201s 192ms/step - accuracy: 0.9032 - loss: 0.3698 - val_accuracy: 0.8965 - val_loss:
0.3779
Epoch 7/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 208s 199ms/step - accuracy: 0.9076 - loss: 0.3556 - val_accuracy: 0.8930 - val_loss:
0.3951
Epoch 8/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 196s 192ms/step - accuracy: 0.9092 - loss: 0.3460 - val_accuracy: 0.8995 - val_loss:
0.3715
Epoch 9/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 168s 200ms/step - accuracy: 0.9120 - loss: 0.3386 - val_accuracy: 0.9088 - val_loss:
0.3490
Epoch 10/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 196s 192ms/step - accuracy: 0.9133 - loss: 0.3360 - val_accuracy: 0.9037 - val_loss:
0.3564
313/313 ━━━━━━━━━━━━━━━━━━━━ 8s 25ms/step - accuracy: 0.9015 - loss: 0.3662
Test Loss: 0.36276623606681824
Test Accuracy: 0.9021999835968018
313/313 ━━━━━━━━━━━━━━━━━━━━ 9s
28ms/step
```



Experiment 4.

4) CNN Model with Dropout

```
import numpy as np
import matplotlib.pyplot as plt
from keras.datasets import fashion_mnist
from keras.models import Sequential
from keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Activation, Dropout
from keras.utils import to_categorical

# Load and preprocess the data
(train_X, train_Y), (test_X, test_Y) = fashion_mnist.load_data()

# Reshape and normalize
train_X = train_X.reshape(-1, 28, 28, 1).astype('float32') / 255.0
test_X = test_X.reshape(-1, 28, 28, 1).astype('float32') / 255.0

# One-hot encode the labels
```

```
train_Y_one_hot = to_categorical(train_Y, 10)
test_Y_one_hot = to_categorical(test_Y, 10)

# Build CNN model with Dropout (no regularization)
model = Sequential()

# 1st Conv Layer
model.add(Conv2D(32, (3, 3), padding='same', input_shape=(28, 28, 1)))
model.add(Activation('relu'))
model.add(MaxPooling2D(pool_size=(2, 2)))

# 2nd Conv Layer
model.add(Conv2D(64, (3, 3), padding='same'))
model.add(Activation('relu'))
model.add(MaxPooling2D(pool_size=(2, 2)))

# 3rd Conv Layer
model.add(Conv2D(128, (3, 3), padding='same'))
model.add(Activation('relu'))

# 4th Conv Layer
model.add(Conv2D(128, (3, 3), padding='same'))
model.add(Activation('relu'))

# Flatten and Dense layers
model.add(Flatten())
model.add(Dense(128, activation='relu'))
model.add(Dropout(0.5))  # Dropout only here
model.add(Dense(10, activation='softmax'))

# Compile the model
model.compile(optimizer='adam',
              loss='categorical_crossentropy',
              metrics=['accuracy'])

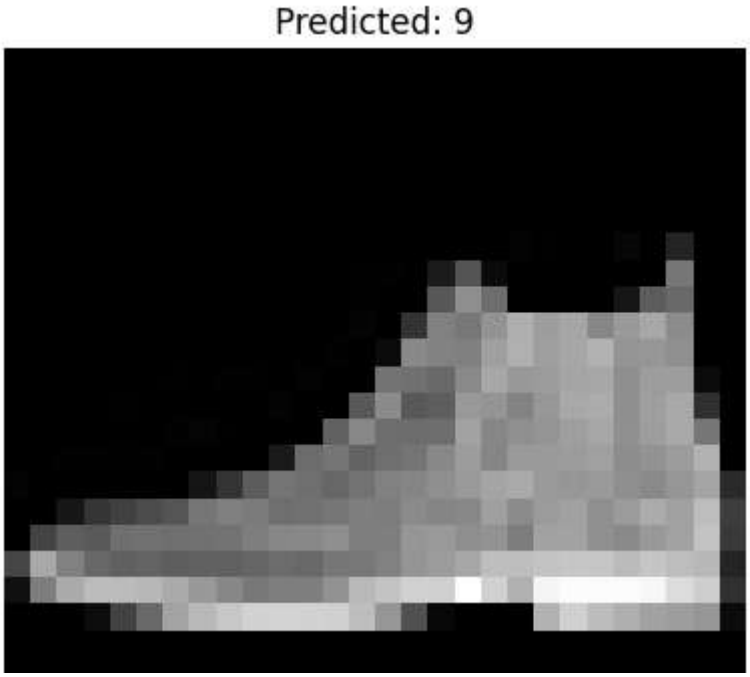
# Train the model
model.fit(train_X, train_Y_one_hot, epochs=10, batch_size=64, validation_split=0.1)

# Evaluate the model
test_loss, test_acc = model.evaluate(test_X, test_Y_one_hot)
print("Test Loss:", test_loss)
print("Test Accuracy:", test_acc)

# Predict and visualize
```

```
predictions = model.predict(test_X)
plt.imshow(test_X[0].reshape(28, 28), cmap='gray')
plt.title(f"Predicted: {np.argmax(predictions[0])}")
plt.axis('off')
plt.show()
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/train-labels-idx1-ubyte.gz
29515/29515 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/train-images-idx3-ubyte.gz
26421880/26421880 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/t10k-labels-idx1-ubyte.gz
5148/5148 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/t10k-images-idx3-ubyte.gz
4422102/4422102 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
/usr/local/lib/python3.11/dist-packages/keras/src/layers/convolutional/base_conv.py:107: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Epoch 1/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 165s 191ms/step - accuracy: 0.7250 - loss: 0.7624 - val_accuracy: 0.8762 - val_loss: 0.3254
Epoch 2/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 199s 188ms/step - accuracy: 0.8800 - loss: 0.3373 - val_accuracy: 0.9052 - val_loss: 0.2653
Epoch 3/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 202s 189ms/step - accuracy: 0.9028 - loss: 0.2755 - val_accuracy: 0.9100 - val_loss: 0.2465
Epoch 4/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 159s 189ms/step - accuracy: 0.9138 - loss: 0.2375 - val_accuracy: 0.9125 - val_loss: 0.2372
Epoch 5/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 159s 189ms/step - accuracy: 0.9246 - loss: 0.2107 - val_accuracy: 0.9193 - val_loss: 0.2213
Epoch 6/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 202s 188ms/step - accuracy: 0.9314 - loss: 0.1916 - val_accuracy: 0.9197 - val_loss: 0.2306
Epoch 7/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 158s 187ms/step - accuracy: 0.9364 - loss: 0.1747 - val_accuracy: 0.9225 - val_loss: 0.2208
Epoch 8/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 204s 189ms/step - accuracy: 0.9431 - loss: 0.1507 - val_accuracy: 0.9233 - val_loss: 0.2198
Epoch 9/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 205s 193ms/step - accuracy: 0.9507 - loss: 0.1325 - val_accuracy: 0.9220 - val_loss: 0.2260
Epoch 10/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 197s 187ms/step - accuracy: 0.9549 - loss: 0.1196 - val_accuracy: 0.9215 - val_loss: 0.2629
313/313 ━━━━━━━━━━━━━━━━━━━━ 9s 30ms/step - accuracy: 0.9179 - loss: 0.2792
Test Loss: 0.27352845668792725
Test Accuracy: 0.9192000031471252
313/313 ━━━━━━━━━━━━━━━━━━━━ 9s 29ms/step
```



Experiment 4.

5.CNN Model with L2 (or L1) regularization and Dropout (combined)

```
import numpy as np
import matplotlib.pyplot as plt
from keras.datasets import fashion_mnist
from keras.models import Sequential
from keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Activation, Dropout
from keras.utils import to_categorical
from keras import regularizers

# Load and preprocess the data
(train_X, train_Y), (test_X, test_Y) = fashion_mnist.load_data()
train_X = train_X.reshape(-1, 28, 28, 1).astype('float32') / 255.0
test_X = test_X.reshape(-1, 28, 28, 1).astype('float32') / 255.0

# One-hot encode the labels
train_Y_one_hot = to_categorical(train_Y, 10)
test_Y_one_hot = to_categorical(test_Y, 10)

# Set L2 regularization strength
l2_strength = 0.001
```

```
# Build the model
model = Sequential()

# 1st Conv Layer
model.add(Conv2D(32, (3, 3), padding='same', input_shape=(28, 28, 1),
          kernel_regularizer=regularizers.l2(l2_strength)))
model.add(Activation('relu'))
model.add(MaxPooling2D(pool_size=(2, 2)))

# 2nd Conv Layer
model.add(Conv2D(64, (3, 3), padding='same',
          kernel_regularizer=regularizers.l2(l2_strength)))
model.add(Activation('relu'))
model.add(MaxPooling2D(pool_size=(2, 2)))

# 3rd Conv Layer
model.add(Conv2D(128, (3, 3), padding='same',
          kernel_regularizer=regularizers.l2(l2_strength)))
model.add(Activation('relu'))

# 4th Conv Layer
model.add(Conv2D(128, (3, 3), padding='same',
          kernel_regularizer=regularizers.l2(l2_strength)))
model.add(Activation('relu'))

# Flatten + Dense
model.add(Flatten())
model.add(Dense(128, activation='relu',
          kernel_regularizer=regularizers.l2(l2_strength)))
model.add(Dropout(0.5))  # Dropout here after dense layer
model.add(Dense(10, activation='softmax'))

# Compile model
model.compile(optimizer='adam',
          loss='categorical_crossentropy',
          metrics=['accuracy'])

# Train model
model.fit(train_X, train_Y_one_hot, epochs=10, batch_size=64, validation_split=0.1)

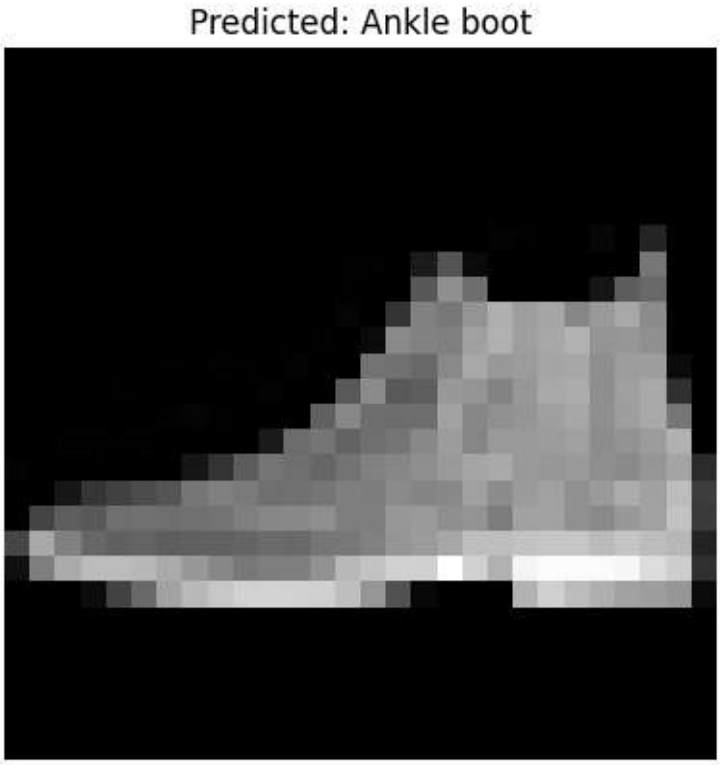
# Evaluate
test_loss, test_acc = model.evaluate(test_X, test_Y_one_hot)
print("Test Loss:", test_loss)
print("Test Accuracy:", test_acc)
```

```
# Predict and visualize
predictions = model.predict(test_X)
labels = ['T-shirt/top', 'Trouser', 'Pullover', 'Dress', 'Coat',
        'Sandal', 'Shirt', 'Sneaker', 'Bag', 'Ankle boot']
predicted_class = np.argmax(predictions[0])
print(f"Predicted label: {predicted_class} ({labels[predicted_class]})")

plt.imshow(test_X[0].reshape(28, 28), cmap='gray')
plt.title(f"Predicted: {labels[predicted_class]}")
plt.axis('off')
plt.show()
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/train-labels-idx1-ubyte.gz
29515/29515 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/train-images-idx3-ubyte.gz
26421880/26421880 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/t10k-labels-idx1-ubyte.gz
5148/5148 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/t10k-images-idx3-ubyte.gz
4422102/4422102 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
/usr/local/lib/python3.11/dist-packages/keras/src/layers/convolutional/base_conv.py:107: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Epoch 1/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 151s 175ms/step - accuracy: 0.7132 - loss: 1.0721 - val_accuracy: 0.8567 - val_loss: 0.5509
Epoch 2/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 201s 174ms/step - accuracy: 0.8546 - loss: 0.5893 - val_accuracy: 0.8747 - val_loss: 0.4868
Epoch 3/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 201s 173ms/step - accuracy: 0.8738 - loss: 0.5218 - val_accuracy: 0.8823 - val_loss: 0.4717
Epoch 4/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 203s 173ms/step - accuracy: 0.8769 - loss: 0.4949 - val_accuracy: 0.8943 - val_loss: 0.4322
Epoch 5/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 148s 175ms/step - accuracy: 0.8840 - loss: 0.4628 - val_accuracy: 0.9003 - val_loss: 0.4030
Epoch 6/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 148s 175ms/step - accuracy: 0.8876 - loss: 0.4467 - val_accuracy: 0.8952 - val_loss: 0.3999
Epoch 7/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 201s 173ms/step - accuracy: 0.8928 - loss: 0.4253 - val_accuracy: 0.9010 - val_loss: 0.3919
Epoch 8/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 203s 175ms/step - accuracy: 0.8955 - loss: 0.4193 - val_accuracy: 0.9013 - val_loss: 0.3861
Epoch 9/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 147s 175ms/step - accuracy: 0.8978 - loss: 0.4112 - val_accuracy: 0.9045 - val_loss: 0.3809
Epoch 10/10
```

```
844/844 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 147s 174ms/step - accuracy: 0.8982 - loss: 0.4005 - val_accuracy: 0.9035 - val_loss:
0.3668
313/313 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 8s 24ms/step - accuracy: 0.8972 - loss: 0.3898
Test Loss: 0.38851872086524963
Test Accuracy: 0.8971999883651733
313/313 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 10s 32ms/step
Predicted label: 9 (Ankle boot)
```



Experiment 5.

Implement a python program to demonstrate the Image captioning.

```
import torch
from transformers import BlipProcessor, BlipForConditionalGeneration
from PIL import Image
import requests
import matplotlib.pyplot as plt

# Load a sample image (you can replace with local path)
#image_url = "https://storage.googleapis.com/sfr-vision-language-research/BLIP/demo.jpg"
```

```
image_url = "https://t4.ftcdn.net/jpg/01/67/29/67/360_F_167296790_tR1y8W6Gm7O7hKYPdByHVWwSRgKCzIFi.jpg"

image = Image.open(requests.get(image_url, stream=True).raw).convert("RGB")

# Display the image
plt.imshow(image)
plt.axis('off')
plt.title("Input Image")
plt.show()

# Load BLIP processor and model
processor = BlipProcessor.from_pretrained("Salesforce/blip-image-captioning-base")
model = BlipForConditionalGeneration.from_pretrained("Salesforce/blip-image-captioning-base")

# Preprocess the image
inputs = processor(images=image, return_tensors="pt")

# Generate caption
with torch.no_grad():
    caption_ids = model.generate(**inputs)
    caption = processor.decode(caption_ids[0], skip_special_tokens=True)

print("Generated Caption:")
print(caption)
```

Output :

```
Drive already mounted at /content/gdrive; to attempt to forcibly remount, call drive.mount("/content/gdrive", force_remount=True).
```



Experiment 6.

Implement a text classification model using Bidirectional-LSTM on a suitable dataset.

```
# Importing necessary libraries
import numpy as np
from tensorflow.keras.datasets import imdb
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, Bidirectional, LSTM, Dense, Dropout
from tensorflow.keras.optimizers import Adam
```

```
# Set random seed for reproducibility
np.random.seed(42)

# Load the IMDB dataset
(x_train, y_train), (x_test, y_test) = imdb.load_data(num_words=10000)  # Use top 10,000 words

# Padding the sequences to have the same length (max length = 500)
max_len = 500
x_train = pad_sequences(x_train, maxlen=max_len)
x_test = pad_sequences(x_test, maxlen=max_len)

# Building the Bidirectional LSTM model
model = Sequential()

# Embedding layer to convert word indices into dense vectors
model.add(Embedding(input_dim=10000, output_dim=128, input_length=max_len))

# Bidirectional LSTM layer
model.add(Bidirectional(LSTM(128, return_sequences=False)))

# Dropout layer to prevent overfitting
model.add(Dropout(0.5))

# Fully connected output layer with a sigmoid activation function for binary classification
model.add(Dense(1, activation='sigmoid'))

# Compile the model
model.compile(optimizer=Adam(), loss='binary_crossentropy', metrics=['accuracy'])

# Summary of the model
model.summary()

# Train the model
history = model.fit(x_train, y_train, epochs=5, batch_size=64, validation_data=(x_test, y_test))

# Evaluate the model
test_loss, test_acc = model.evaluate(x_test, y_test)
print(f"Test accuracy: {test_acc * 100:.2f}%")

# Example prediction on test data
sample_index = 100  # Random test sample
```

prediction = model.predict(np.array([x_test[sample_index]]))
print(f"Prediction for sample {sample_index}: {'Positive' if prediction > 0.5 else 'Negative'}")

Output:

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/imdb.npz
17464789/17464789 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
/usr/local/lib/python3.11/dist-packages/keras/src/layers/core/embedding.py:90: UserWarning: Argument `input_length` is deprecated. Just remove it.
  warnings.warn(
Model: "sequential"
```

Layer (type)	Output Shape	Param #
embedding (Embedding)	?	0 (unbuilt)
bidirectional (Bidirectional)	?	0 (unbuilt)
dropout (Dropout)	?	0
dense (Dense)	?	0 (unbuilt)

```
 Total params: 0 (0.00 B)
 Trainable params: 0 (0.00 B)
 Non-trainable params: 0 (0.00 B)
Epoch 1/5
391/391 ━━━━━━━━━━━━━━━━━━━━ 1099s 3s/step - accuracy: 0.6930 - loss: 0.5511 - val_accuracy: 0.8410 - val_loss: 0.3713
Epoch 2/5
391/391 ━━━━━━━━━━━━━━━━━━━━ 1091s 3s/step - accuracy: 0.8780 - loss: 0.3027 - val_accuracy: 0.8312 - val_loss: 0.3905
Epoch 3/5
391/391 ━━━━━━━━━━━━━━━━━━━━ 1090s 3s/step - accuracy: 0.8842 - loss: 0.2870 - val_accuracy: 0.8738 - val_loss: 0.3130
Epoch 4/5
391/391 ━━━━━━━━━━━━━━━━━━━━ 1097s 3s/step - accuracy: 0.9393 - loss: 0.1696 - val_accuracy: 0.8537 - val_loss: 0.3583
Epoch 5/5
391/391 ━━━━━━━━━━━━━━━━━━━━ 1099s 3s/step - accuracy: 0.9513 - loss: 0.1400 - val_accuracy: 0.8582 - val_loss: 0.3679
782/782 ━━━━━━━━━━━━━━━━━━━━ 350s 448ms/step - accuracy: 0.8569 - loss: 0.3729
Test accuracy: 85.82%
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 488ms/step
Prediction for sample 100: Negative
```

Experiment 7.

Implement Auto encoders for image denoising on MNIST dataset.

```
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow.keras import layers, models

# Load MNIST dataset
(x_train, _), (x_test, _) = tf.keras.datasets.mnist.load_data()

# Normalize to [0,1] and reshape to (batch_size, 28, 28, 1)
x_train = x_train.astype('float32') / 255.
x_test = x_test.astype('float32') / 255.
x_train = np.reshape(x_train, (len(x_train), 28, 28, 1))
x_test = np.reshape(x_test, (len(x_test), 28, 28, 1))

# Define the autoencoder architecture
input_img = layers.Input(shape=(28, 28, 1))

# Encoder
x = layers.Conv2D(16, (3, 3), activation='relu', padding='same')(input_img)
x = layers.MaxPooling2D((2, 2), padding='same')(x)
x = layers.Conv2D(8, (3, 3), activation='relu', padding='same')(x)
encoded = layers.MaxPooling2D((2, 2), padding='same')(x)  # Output shape: (7, 7, 8)

# Decoder
x = layers.Conv2D(8, (3, 3), activation='relu', padding='same')(encoded)
x = layers.UpSampling2D((2, 2))(x)
x = layers.Conv2D(16, (3, 3), activation='relu', padding='same')(x)
x = layers.UpSampling2D((2, 2))(x)
decoded = layers.Conv2D(1, (3, 3), activation='sigmoid', padding='same')(x)
```

```
# Define the autoencoder model
autoencoder = models.Model(input_img, decoded)
autoencoder.compile(optimizer='adam', loss='binary_crossentropy')

# Train the autoencoder
autoencoder.fit(x_train, x_train,
                epochs=10,
                batch_size=128,
                shuffle=True,
                validation_data=(x_test, x_test))

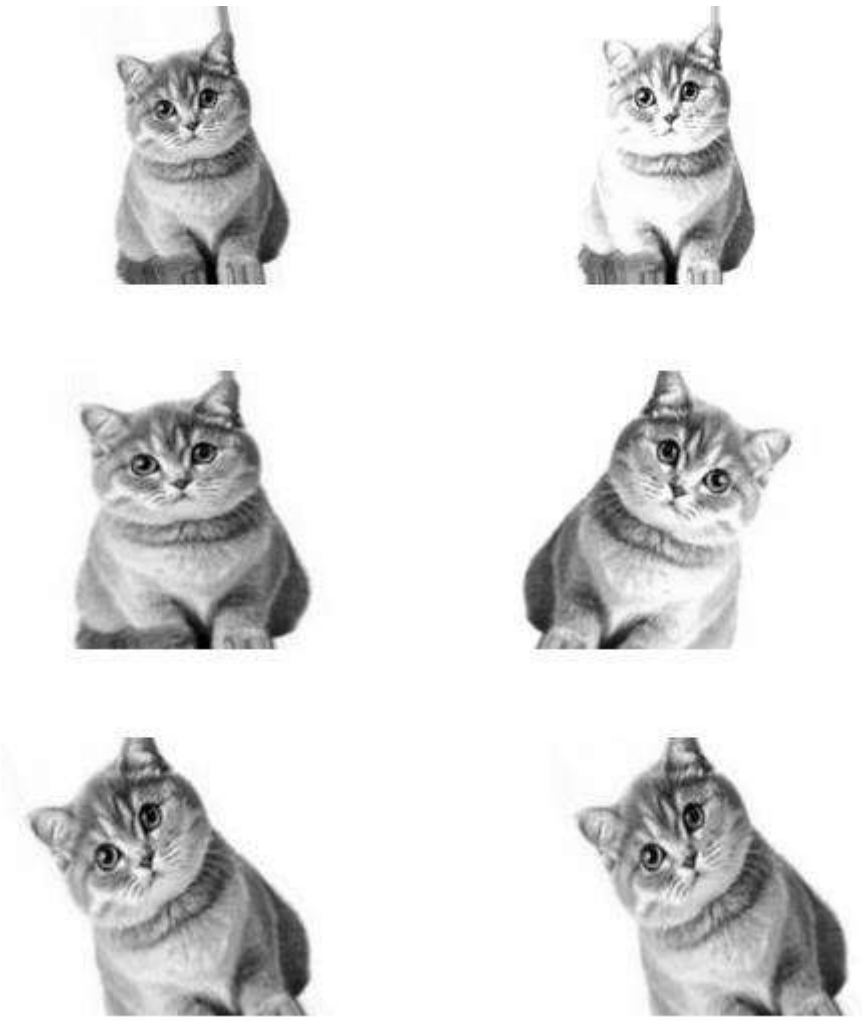
# Reconstruct test images
decoded_imgs = autoencoder.predict(x_test)

# Display original and reconstructed images
n = 10  # Number of images to display
plt.figure(figsize=(20, 4))
for i in range(n):
    # Original image
    ax = plt.subplot(2, n, i + 1)
    plt.imshow(x_test[i].reshape(28, 28), cmap='gray')
    plt.title("Original")
    plt.axis('off')

    # Reconstructed image
    ax = plt.subplot(2, n, i + 1 + n)
    plt.imshow(decoded_imgs[i].reshape(28, 28), cmap='gray')
    plt.title("Reconstructed")
    plt.axis('off')

plt.tight_layout()
plt.show()
```

Output:



Experiment 8.

Implement a Generative Adversarial Network using MNIST dataset to train a model that generates realistic handwritten digits.

```
import torch
import torch.nn as nn
import torch.optim as optim
from torchvision import datasets, transforms
from torch.utils.data import DataLoader
import matplotlib.pyplot as plt
import numpy as np
import torchvision
```

```
# Device configuration
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

# Hyperparameters
latent_size = 64
hidden_size = 256
image_size = 784  # 28x28
batch_size = 100
num_epochs = 10
learning_rate = 0.0002

# MNIST data loader
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5,), (0.5,))  # Scale to [-1, 1]
])
mnist = datasets.MNIST(root='data/', train=True, transform=transform, download=True)
data_loader = DataLoader(dataset=mnist, batch_size=batch_size, shuffle=True)

# Generator network
class Generator(nn.Module):
    def __init__(self):
        super(Generator, self).__init__()
        self.model = nn.Sequential(
            nn.Linear(latent_size, hidden_size),
            nn.LeakyReLU(0.2),
            nn.Linear(hidden_size, hidden_size),
            nn.LeakyReLU(0.2),
            nn.Linear(hidden_size, image_size),
            nn.Tanh()
```

```
        )

    def forward(self, z):
        return self.model(z)

# Discriminator network
class Discriminator(nn.Module):
    def __init__(self):
        super(Discriminator, self).__init__()
        self.model = nn.Sequential(
            nn.Linear(image_size, hidden_size),
            nn.LeakyReLU(0.2),
            nn.Linear(hidden_size, hidden_size),
            nn.LeakyReLU(0.2),
            nn.Linear(hidden_size, 1),
            nn.Sigmoid()
        )

    def forward(self, x):
        return self.model(x)

# Initialize models
G = Generator().to(device)
D = Discriminator().to(device)

# Loss and optimizers
criterion = nn.BCELoss()
d_optimizer = optim.Adam(D.parameters(), lr=learning_rate)
g_optimizer = optim.Adam(G.parameters(), lr=learning_rate)

# Function to denormalize and display images
def denorm(x):
    out = (x + 1) / 2
```

```
    return out.clamp(0, 1)

def show_images(fake_images):
    fake_images = fake_images.reshape(-1, 1, 28, 28)
        grid = torchvision.utils.make_grid(fake_images, nrow=10,
normalize=True)
    plt.figure(figsize=(10, 4))
    plt.imshow(np.transpose(grid.cpu(), (1, 2, 0)))
    plt.axis('off')
    plt.show()

# Training loop
for epoch in range(num_epochs):
    for i, (images, _) in enumerate(data_loader):
        batch_size_curr = images.size(0)
        images = images.reshape(batch_size_curr, -1).to(device)

        # Labels
        real_labels = torch.ones(batch_size_curr, 1).to(device)
        fake_labels = torch.zeros(batch_size_curr, 1).to(device)

        # =========================
        # Train Discriminator
        # =========================
        outputs = D(images)
        d_loss_real = criterion(outputs, real_labels)
        real_score = outputs

        z = torch.randn(batch_size_curr, latent_size).to(device)
        fake_images = G(z)
        outputs = D(fake_images.detach())
        d_loss_fake = criterion(outputs, fake_labels)
        fake_score = outputs
```

```
        d_loss = d_loss_real + d_loss_fake
        D.zero_grad()
        d_loss.backward()
        d_optimizer.step()

        # =========================
        # Train Generator
        # =========================
        z = torch.randn(batch_size_curr, latent_size).to(device)
        fake_images = G(z)
        outputs = D(fake_images)
        g_loss = criterion(outputs, real_labels)  # Trick discriminator

        G.zero_grad()
        g_loss.backward()
        g_optimizer.step()

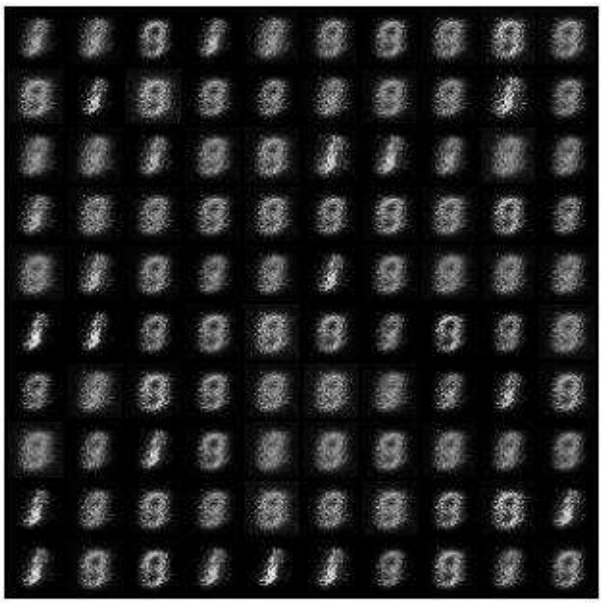
 print(f"Epoch [{epoch+1}/{num_epochs}], d_loss: {d_loss.item():.4f}, g_loss: {g_loss.item():.4f}")

    if (epoch + 1) % 10 == 0:
        with torch.no_grad():
            z = torch.randn(100, latent_size).to(device)
            fake_images = G(z)
            show_images(fake_images)

print("Training completed.")
```

```
100%|██████████| 9.91M/9.91M [00:00<00:00, 37.7MB/s]
100%|██████████| 28.9k/28.9k [00:00<00:00, 19.0MB/s]
100%|██████████| 1.65M/1.65M [00:00<00:00, 49.7MB/s]
100%|██████████| 4.54k/4.54k [00:00<00:00, 3.44MB/s]
```

```
Epoch [1/10], d_loss: 0.2218, g_loss: 2.6849
Epoch [2/10], d_loss: 0.6776, g_loss: 2.6160
Epoch [3/10], d_loss: 0.4259, g_loss: 2.4124
Epoch [4/10], d_loss: 0.5921, g_loss: 2.3435
Epoch [5/10], d_loss: 1.1149, g_loss: 2.7133
Epoch [6/10], d_loss: 0.3931, g_loss: 3.2309
Epoch [7/10], d_loss: 0.8669, g_loss: 2.3428
Epoch [8/10], d_loss: 0.5484, g_loss: 2.9376
Epoch [9/10], d_loss: 0.9649, g_loss: 2.1122
Epoch [10/10], d_loss: 0.3317, g_loss: 2.3034
```



Experiment 9.

Implement a Python program to build a model for face recognition using Python and Open-CV.

```
from google.colab.patches import cv2_imshow
import cv2
import numpy as np
from IPython.display import display, HTML
import time
from PIL import Image
import io
import base64

# Load the cascade for face detection
```

```
face_cascade = cv2.CascadeClassifier(cv2.data.haarcascades + 'haarcascade_frontalface_default.xml')

# HTML and JavaScript to capture webcam frame
html = """
<video width="640" height="480" autoplay></video>
<button onclick="captureImage()">Capture Image</button>
<script>
  let video = document.querySelector('video');
  navigator.mediaDevices.getUserMedia({ video: true })
    .then(function(stream) {
      video.srcObject = stream;
    });

  function captureImage() {
    let canvas = document.createElement('canvas');
    canvas.width = 640;
    canvas.height = 480;
    let context = canvas.getContext('2d');
    context.drawImage(video, 0, 0, 640, 480);
    let dataURL = canvas.toDataURL('image/jpeg');
    google.colab.kernel.invokeFunction('notebook.capture_webcam_image', [dataURL], {});
  }
</script>
"""
display(HTML(html))

# This function will receive the image from JavaScript (via the button click)
def capture_webcam_image(image_data):
    # Decode the base64 image string to bytes
    img_data = image_data.split(',')[1]
    img_bytes = base64.b64decode(img_data)

    # Convert to numpy array
    img = Image.open(io.BytesIO(img_bytes))
    frame = np.array(img)

    # Convert the image to grayscale for face detection
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

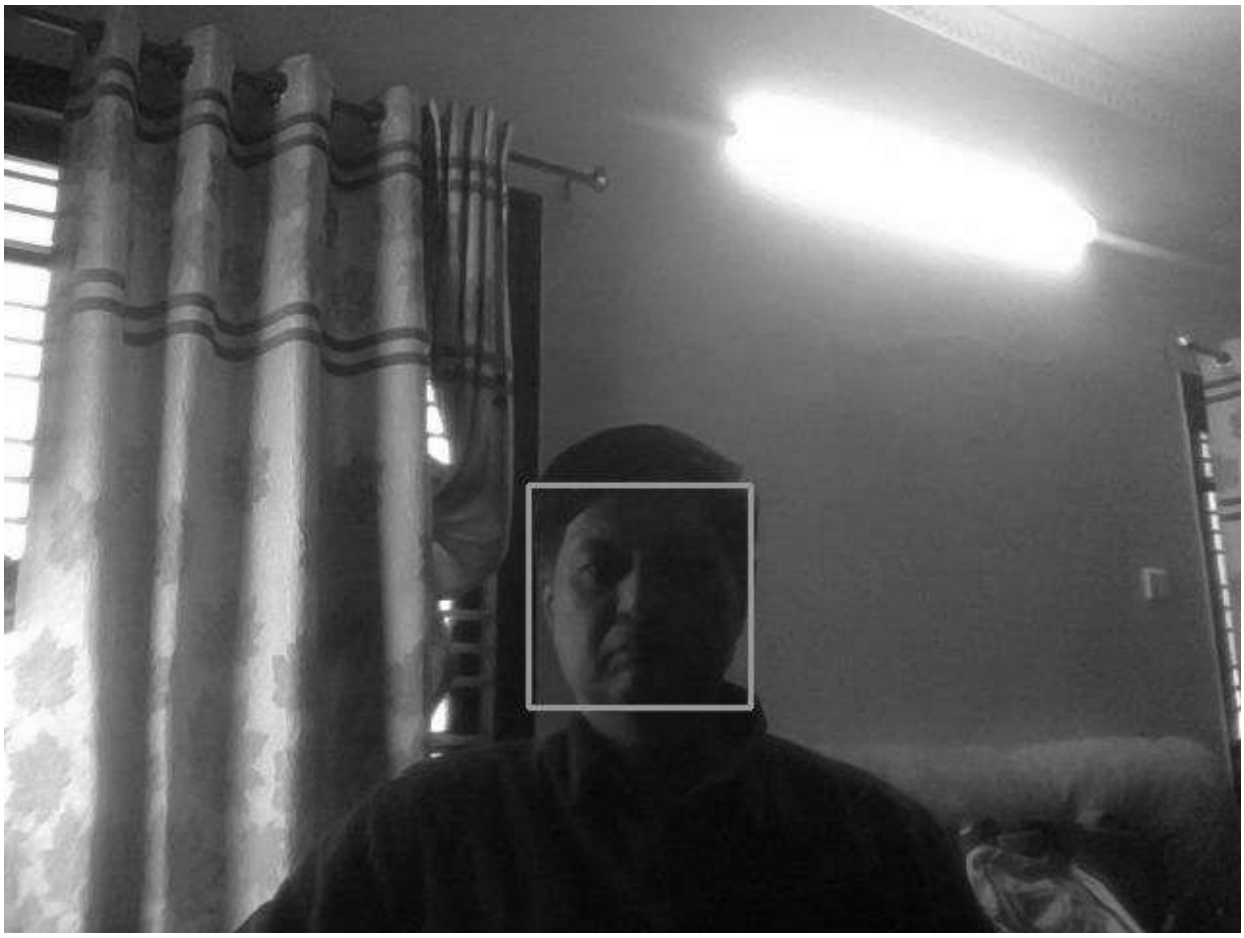
    # Detect faces in the frame
    faces = face_cascade.detectMultiScale(gray, scaleFactor=1.1, minNeighbors=4)

    # Draw rectangles around the detected faces
```

```
    for (x, y, w, h) in faces:
        cv2.rectangle(frame, (x, y), (x + w, y + h), (0, 255, 0), 2)

    # Display the result
    cv2_imshow(frame)

# Register the function in JavaScript that will send image data to Python
from google.colab import output
output.register_callback('notebook.capture_webcam_image', capture_webcam_image)
```



Experiment 10.

Implement an RNN for sentiment analysis using IMDB movie dataset.

```
import tensorflow as tf
from tensorflow.keras.datasets import imdb
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, SimpleRNN, Dense

# -------------------------
# 1. Load and Preprocess Data
# -------------------------
vocab_size = 10000
maxlen = 500
embedding_dim = 32

# Load dataset
(x_train, y_train), (x_test, y_test) = imdb.load_data(num_words=vocab_size)

# Pad sequences to uniform length
x_train = pad_sequences(x_train, maxlen=maxlen)
x_test = pad_sequences(x_test, maxlen=maxlen)

# -------------------------
# 2. Build RNN Model
# -------------------------
model = Sequential([
    Embedding(input_dim=vocab_size, output_dim=embedding_dim, input_length=maxlen),
    SimpleRNN(64),
    Dense(1, activation='sigmoid')
])

# Compile model
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])

# Model summary
model.summary()

# -------------------------
# 3. Train Model
# -------------------------
```

```
model.fit(x_train, y_train, epochs=5, batch_size=64, validation_split=0.2)

# Evaluate
loss, acc = model.evaluate(x_test, y_test)
print(f"Test Accuracy: {acc * 100:.2f}%")

# ------------------------
# 4. Predict a New Sample
# ------------------------

# Get word index
word_index = imdb.get_word_index()
word_index = {k: (v + 3) for k, v in word_index.items()}  # Shift indices for reserved tokens
word_index["<PAD>"] = 0
word_index["<START>"] = 1
word_index["<UNK>"] = 2
word_index["<UNUSED>"] = 3

# Encode a review
def encode_review(text):
    tokens = text.lower().split()
    encoded = [1]  # <START>
    for word in tokens:
        encoded.append(word_index.get(word, 2))  # 2 = <UNK>
    return encoded

# Preprocess and pad
def preprocess_input(text, maxlen=500):
    encoded = encode_review(text)
    padded = pad_sequences([encoded], maxlen=maxlen)
    return padded

# Sample review
sample_review = "This movie was fantastic! The story was compelling and the acting was
great."

# Preprocess and predict
processed_review = preprocess_input(sample_review, maxlen)
prediction = model.predict(processed_review)

# Interpret result
sentiment = "Positive 😀" if prediction[0][0] > 0.5 else "Negative 😞"
print(f"\nSample review: {sample_review}")
print(f"Predicted Sentiment: {sentiment} ({prediction[0][0]:.4f})")
```

Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/imdb.npz

17464789/17464789 ━━━━━━━━━━━━━━━━━━━━ 1s 0us/step

/usr/local/lib/python3.11/dist-packages/keras/src/layers/core/embedding.py:90: UserWarning: Argument `input_length` is deprecated. Just remove it.
warnings.warn(

Model: "sequential"

Layer (type)	Output Shape	Param #
embedding (Embedding)	?	0 (unbuilt)
simple_rnn (SimpleRNN)	?	0 (unbuilt)
dense (Dense)	?	0 (unbuilt)

Total params: 0 (0.00 B)
Trainable params: 0 (0.00 B)
Non-trainable params: 0 (0.00 B)

Epoch 1/5

313/313 ━━━━━━━━━━━━━━━━━━━━ 55s 167ms/step - accuracy: 0.5878 - loss: 0.6480 - val_accuracy: 0.7364 - val_loss: 0.5342

Epoch 2/5

313/313 ━━━━━━━━━━━━━━━━━━━━ 87s 182ms/step - accuracy: 0.8247 - loss: 0.4124 - val_accuracy: 0.8316 - val_loss: 0.3959

Epoch 3/5

313/313 ━━━━━━━━━━━━━━━━━━━━ 59s 188ms/step - accuracy: 0.9041 - loss: 0.2530 - val_accuracy: 0.5174 - val_loss: 1.6030

Epoch 4/5

313/313 ━━━━━━━━━━━━━━━━━━━━ 81s 184ms/step - accuracy: 0.7671 - loss: 0.4912 - val_accuracy: 0.7120 - val_loss: 0.6216
Epoch 5/5
313/313 ━━━━━━━━━━━━━━━━━━━━ 60s 191ms/step - accuracy: 0.9677 - loss: 0.1031 - val_accuracy: 0.7782 - val_loss: 0.5897
782/782 ━━━━━━━━━━━━━━━━━━━━ 29s 37ms/step - accuracy: 0.7755 - loss: 0.5933
Test Accuracy: 77.52%
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/imdb_word_index.json
1641221/1641221 ━━━━━━━━━━━━━━━━━━━━ 1s 0us/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 240ms/step

Sample review: This movie was fantastic! The story was compelling and the acting was great.
Predicted Sentiment: Positive 😀 (0.5120)

VIVA QUESTIONS

5) What is the difference between machine learning and deep learning?
6) What is a neural network, and how does it learn?
7) Explain the concept of backpropagation. How are gradients calculated?
8) What are some commonly used activation functions, and what are their pros and cons?
9) What is the vanishing gradient problem, and how can it be addressed?
10) How does batch normalization work, and why is it useful?
11) What does the term 'convolution' refer to in CNNs?
12) What is the primary purpose of pooling layers in CNNs?
13) Which of the following is commonly used as an activation function in CNNs?
14) What is the role of filters (or kernels) in a CNN?
15) What does a stride of 2 in a convolutional layer do?
16) How does padding help in convolutional layers?
17) Why is ReLU preferred over Sigmoid in CNNs?
18) Compare optimizers like SGD, Adam, and RMSProp.
19) What are the key components of a neural network layer?
20) Compare feedforward neural networks (FNNs) and recurrent neural networks (RNNs).

21) What are convolutional neural networks (CNNs), and why are they useful for image data?
22) Which of the following techniques is commonly used to prevent overfitting in CNNs?
23) What is the key feature of an RNN compared to a traditional neural network?
24) Which type of data is RNN particularly suited for?
25) Which activation function is commonly used in the hidden layers of RNNs?
26) In a simple RNN, what is shared across time steps?
27) What are the main components of an LSTM cell?
28) How does a GRU differ from an LSTM?
29) In sequence-to-sequence models with RNNs, what is the role of the encoder?
30) What are attention mechanisms and how do they differ from recurrent architecture?
31) What is overfitting in deep learning, and how can it be prevented?
32) Explain dropout and how it works as a regularization technique.
33) What is early stopping, and how does it help in training?
34) What are autoencoders, and where are they used?
35) What are GANs (Generative Adversarial Networks), and how do they work?
36) Explain the Transformer architecture and its advantages.
37) How do you evaluate the performance of a deep learning model? Mention metrics for classification and regression.
38) Give some real-world applications of deep learning in various domains.
39) What are the ethical concerns and limitations of deep learning models?